PROJECT REPORT

Event-Driven E-Commerce

Order Management & Pricing Engine

Developer	**Team Project**
Stack	TypeScript · Express.js · MongoDB
Total Commits	50 commits — 100% sole contributor
Test Coverage	36 unit tests — all passing
Design Patterns	8 core patterns implemented

github.com/vks-g/Event-Driven-E-Commerce-Order-Management-Pricing-Engine

1. Executive Summary

This report documents the design, architecture, and implementation of the Event-Driven E-Commerce Order Management and Pricing Engine — a solo backend engineering project built with TypeScript, Express.js, and MongoDB.

The system solves a core problem in e-commerce backends: tight coupling between order management, inventory tracking, and pricing logic makes systems fragile and difficult to extend. This project addresses that by introducing a fully event-driven architecture where modules communicate exclusively through a central EventBus, ensuring each module is independently deployable and testable.

The implementation spans 50 commits across four functional modules — Inventory, Pricing, Orders, and Payments — with 8 design patterns applied deliberately to address real engineering trade-offs, supported by 36 unit tests covering all critical paths.

Metric	Value
Architecture Style	Event-Driven with synchronous REST API layer
Language	TypeScript (strict mode)
Framework	Express.js
Database	MongoDB with Mongoose ODM
Total Source Files	~35 TypeScript source files
Total Commits	50 (sole contributor)
Unit Tests	36 passing
Design Patterns	8 (Observer, Strategy, Decorator, Adapter, State Machine, Repository, Singleton, Middleware Chain)
API Endpoints	18 endpoints across 4 modules

2. Problem Statement & Motivation

Traditional monolithic e-commerce backends typically implement order management, inventory, and pricing in tightly coupled service layers. This creates several engineering problems:

- A price rule change requires modifying both the pricing module and the order service that calls it.
- Order cancellations require explicit service-to-service calls to release inventory, creating fragile dependencies.

- Adding new features — such as email notifications or analytics events on order creation — requires modifying existing, tested code.
- Race conditions during simultaneous stock reservation can result in overselling inventory.
- Duplicate API requests (e.g., network retry on order creation) can create duplicate orders.

This project was built to demonstrate how event-driven architecture and established design patterns directly resolve each of these problems, producing a system that is loosely coupled, open to extension, and resilient to operational failures.

3. System Architecture

3.1 High-Level Architecture

The system is structured in five horizontal layers: the client-facing REST API, a middleware pipeline, four domain modules, a shared EventBus, and the MongoDB data layer. The critical design decision is that domain modules never import or call each other — all cross-module communication passes through the EventBus.

Architecture Principle

"No service imports another service. OrderService does not know InventoryService exists. It emits an event and the system reacts. Adding a new reaction to an order being created — say, sending an email — requires writing one new handler and registering it. Zero changes to any existing code. This is the Open/Closed Principle in practice."

3.2 Module Breakdown

Module	Responsibility	Key Components
Inventory	Product catalogue, stock levels, atomic reservation and release	inventoryModel, inventoryRepository, inventoryService, inventoryController
Pricing	Runtime pricing strategy selection, composable discount stacking	4 Strategy classes, 4 Decorator classes, pricingService
Orders	Order lifecycle from creation through delivery, state enforcement	orderStateMachine, orderService, orderRepository, idempotencyModel
Payments	Payment provider abstraction (stubbed for extension)	PaymentAdapter (abstract), StripeAdapter (stub), paymentService

3.3 Request Pipeline

Every inbound HTTP request passes through the following middleware chain before reaching a controller:

- **1.** CORS — Cross-origin resource sharing headers.
- **2.** JSON Parser — Body deserialization.
- **3.** Rate Limiter — 100 requests per 15 minutes per IP.
- **4.** Idempotency Check — For write endpoints, checks for a duplicate Idempotency-Key header and returns the cached response if found.
- **5.** Joi Validation — Schema validation on the request body; returns structured 400 errors on failure.
- **6.** Controller — Calls the service layer and formats the response via responseFormatter.
- **7.** Centralized Error Handler — Catches all thrown errors and returns a consistent { success, data, error } envelope.

3.4 Data Layer

MongoDB was selected for its flexible document model, which accommodates variable order item structures and discount arrays without schema migrations. Three collections are in use:

Collection	Purpose	Key Fields
inventory	Product catalogue and stock levels	sku (unique), stock, reservedStock, version (optimistic lock), basePrice
orders	Order documents and lifecycle state	items[], status, totalPrice, subtotal, discountsApplied[], idempotencyKey (unique)
idempotencykeys	Cached responses for duplicate requests	key (unique), response, statusCode, TTL: 24 hours

4. Event-Driven Core: The EventBus

4.1 Design

The EventBus is implemented as a Singleton — a single shared instance exists for the application lifetime. It maintains a registry of event-name-to-handler mappings. Any module can emit an event (producer) or register a handler for an event (consumer). Producers and consumers are completely decoupled; neither knows the other exists.

Event Name	Emitted By	Handled By	Effect
ORDER_CREATED	OrderService	InventoryHandler	Reserves stock for each item in the order

Event Name	Emitted By	Handled By	Effect
INVENTORY_RESERVED	InventoryHandler	OrderHandler	Transitions order from PENDING to CONFIRMED
ORDER_CONFIRMED	OrderHandler	—	Terminal event (extensible for notifications, analytics)
ORDER_CANCELLED	OrderService	InventoryHandler	Releases previously reserved stock (compensating transaction)
INVENTORY_RELEASED	InventoryHandler	—	Terminal event confirming stock restoration

4.2 Order Creation Flow

The order creation flow is the primary happy path through the system and demonstrates the event-driven pattern end-to-end:

- **Step 1:** Client sends POST /api/orders with items array and pricing strategy.
- **Step 2:** OrderService calls PricingService to calculate the cart total with chosen strategy and any discounts.
- **Step 3:** OrderService saves the order to MongoDB with status PENDING.
- **Step 4:** OrderService emits ORDER_CREATED event and immediately returns 201 to the client. The order is now in an eventually-consistent state.
- **Step 5:** InventoryHandler receives ORDER_CREATED. It calls InventoryService.reserveStock() for each item using an atomic update with optimistic locking.
- **Step 6:** InventoryHandler emits INVENTORY_RESERVED.
- **Step 7:** OrderHandler receives INVENTORY_RESERVED. It calls OrderService.confirmOrder() which transitions the order PENDING → CONFIRMED.
- **Step 8:** OrderHandler emits ORDER_CONFIRMED. The order is now fully confirmed and inventory is committed.

4.3 Cancellation & Compensating Transactions

When an order is cancelled, the EventBus enables a compensating transaction pattern to restore system consistency:

- **Step 1:** Client sends POST /api/orders/:id/cancel.
- **Step 2:** OrderService validates the transition via the State Machine and updates status to CANCELLED.
- **Step 3:** OrderService emits ORDER_CANCELLED with the full items array.

- **Step 4:** InventoryHandler receives ORDER_CANCELLED. It calls InventoryService.releaseStock() for each item, atomically decrementing reservedStock and incrementing stock.
- **Step 5:** InventoryHandler emits INVENTORY_RELEASED confirming the compensating transaction is complete.

This pattern guarantees inventory is never permanently consumed by a cancelled order, without requiring a two-phase commit or distributed transaction coordinator.

5. Design Patterns

5.1 Pattern Overview

Pattern	Location	SOLID Principle Applied	Problem Solved
Observer	events/EventBus.ts	Open/Closed Principle	Modules react to events without being coupled to producers
Strategy	pricing/strategies/	Open/Closed Principle	Swap pricing algorithm at runtime without touching consumers
Decorator	pricing/decorators/	Open/Closed + Single Responsibility	Stack discounts composably without combinatorial subclassing
Adapter	payments/adapters/	Dependency Inversion Principle	Abstract payment providers behind a uniform interface
State Machine	orders/orderStateMachine.ts	Single Responsibility	Enforce valid order lifecycle transitions, prevent data corruption
Repository	*Repository.ts files	Dependency Inversion Principle	Isolate data access from business logic
Singleton	events/EventBus.ts	—	One shared EventBus instance across the entire application
Middleware Chain	middleware/	Single Responsibility	Cross-cutting concerns separated from domain logic

5.2 Strategy Pattern — Pricing

The Pricing module implements the Strategy pattern to allow runtime selection of pricing algorithms. The abstract PricingStrategy base class defines a single calculatePrice(basePrice, context) contract. Four concrete strategies are implemented:

Strategy	Logic	Example
RegularPricing	Returns base price unchanged	100 units at $10 → $10 each
SeasonalPricing	Applies multiplier based on current season (summer +15%, holiday +25%, off-season -10%)	Holiday season: $10 → $12.50
BulkPricing	Tiered volume discounts: 10+ items = 5% off, 50+ = 15% off, 100+ = 25% off	100 units at $10 → $7.50 each
MemberPricing	Member tier discounts: Silver 5%, Gold 10%, Platinum 15%	Platinum member: $10 → $8.50

Strategies are selected at call time by passing a string identifier (e.g., strategy: "bulk") to PricingService. The service maps the string to the concrete class. Adding a new strategy requires zero changes to OrderService or PricingController.

5.3 Decorator Pattern — Discounts

The Decorator pattern enables composable, stackable discount application. Each discount decorator wraps a PricingStrategy (or another decorator) and modifies the result. Four decorators are implemented:

Decorator	Behaviour	Edge Case Handled
PercentageDiscount	Reduces final price by a percentage (e.g., 10% off)	Applied after strategy pricing
FlatDiscount	Reduces final price by a fixed amount (e.g., $5 off)	Price floors at $0, never goes negative
CouponDiscount	Validates a coupon code against a registry, applies its discount	Invalid codes are silently skipped
LoyaltyDiscount	Applies discount based on loyalty tier (bronze through diamond)	Stacks after other decorators

A cart calculation can stack all four: MemberPricing → PercentageDiscount → CouponDiscount → LoyaltyDiscount. Each layer reads the previous result and adds its effect. The discountsApplied array on the order record logs every discount applied, for auditability.

5.4 State Machine — Order Lifecycle

The OrderStateMachine defines all valid order statuses and the legal transitions between them. Every call to OrderService that changes status is routed through the state machine. An illegal transition (e.g., PENDING → DELIVERED) throws an error, preventing data corruption from malformed API calls.

From Status	Valid Next Statuses
PENDING	CONFIRMED, CANCELLED
CONFIRMED	PAYMENT_PROCESSING, CANCELLED
PAYMENT_PROCESSING	PAID, PAYMENT_FAILED
PAYMENT_FAILED	PENDING (retry), CANCELLED
PAID	SHIPPING, REFUNDED
SHIPPING	SHIPPED
SHIPPED	DELIVERED
CANCELLED / DELIVERED / REFUNDED	None (terminal states)

5.5 Adapter Pattern — Payments

The PaymentAdapter abstract class defines a uniform interface: processPayment(), refundPayment(), and getStatus(). The StripeAdapter implements this interface as a stub (returning mock success responses). To integrate a real payment provider or switch from Stripe to another gateway, only the adapter file needs to be written — no changes to PaymentService, routes, or controllers.

6. Key Engineering Decisions

6.1 Optimistic Locking on Inventory

The inventory schema includes a version field (integer). Every reservation or release performs an atomic MongoDB update with a version check:

Optimistic Locking Pattern

```
findOneAndUpdate({ sku, version: currentVersion, stock: { $gte: qty } },
  { $inc: { stock: -qty, reservedStock: +qty, version: +1 } })
```

If the version has changed (concurrent update), the query returns null.
The service retries or throws InsufficientStockError.

This prevents overselling without requiring pessimistic database locks, which would serialize all inventory operations and create a bottleneck under load.

6.2 Idempotency Middleware

The idempotencyMiddleware reads the Idempotency-Key header on write requests. If a matching key exists in the idempotencykeys collection, it returns the cached response immediately — the handler is never invoked. If the key is new, the request is processed normally and the response is stored with a 24-hour TTL.

This ensures that network retries on POST /api/orders never create duplicate orders, which is critical for any payment-adjacent workflow where at-most-once semantics are required.

6.3 Eventual Consistency Model

Order creation returns 201 to the client as soon as the order is saved as PENDING — before inventory is reserved or the order is confirmed. The inventory reservation and confirmation happen asynchronously via the EventBus. This means:

- Response latency for the client is decoupled from inventory processing time.
- The system can absorb spikes in order creation without blocking.
- A client polling GET /api/orders/:id will see PENDING briefly, then CONFIRMED once the event chain completes.

This is an intentional trade-off: strong consistency is sacrificed for better throughput, consistent with how production e-commerce systems (e.g., Amazon) operate.

6.4 Structured Error Handling

All errors are funnelled through a centralized errorHandler middleware. Custom error classes (e.g., InsufficientStockError, InvalidStateTransitionError) carry HTTP status codes. The response envelope is always { success: boolean, data: object | null, error: string | null }, making API responses predictable for consumers.

6.5 Winston Structured Logging

Winston is configured as the application logger with log levels mapped to environment (debug in development, info/error in production). All service operations log structured JSON entries, making logs queryable in production observability tools like Datadog or CloudWatch.

7. API Reference

7.1 Inventory Endpoints

Method	Endpoint	Description	Auth
GET	/api/inventory	List all products with stock levels	None
GET	/api/inventory/:sku	Get a single product by SKU	None
POST	/api/inventory	Add a new product to the catalogue	None
PATCH	/api/inventory/:sku/stock	Update available stock for a product	None

7.2 Pricing Endpoints

Method	Endpoint	Description	Auth
POST	/api/pricing/calculate	Calculate cart total with chosen strategy and discounts	None
POST	/api/pricing/simulate	Compare all four strategies side-by-side for a given cart	None
GET	/api/pricing/strategies	List available strategies and their descriptions	None

7.3 Order Endpoints

Method	Endpoint	Description	Notes
POST	/api/orders	Create a new order	Supports Idempotency-Key header
GET	/api/orders	List all orders	—
GET	/api/orders/:id	Get a single order by ID	—
POST	/api/orders/:id/confirm	Manually confirm an order	PENDING → CONFIRMED
POST	/api/orders/:id/cancel	Cancel an order	Triggers INVENTORY_RELEASED event
POST	/api/orders/:id/transition	Transition to any valid next status	State machine validated
GET	/api/orders/transitions	View all valid state transitions	Returns full transition map

7.4 Payment Endpoints (Stubbed)

Method	Endpoint	Status	Notes
POST	/api/payments/process	501 Not Implemented	StripeAdapter ready for real implementation
POST	/api/payments/:id/refund	501 Not Implemented	Adapter interface defined
GET	/api/payments/:id/status	501 Not Implemented	Returns mock status object

8. Testing

8.1 Test Coverage Summary

The test suite contains 36 unit tests across three test files, all passing. Tests are colocated with their subject in tests/unit/ and run via Jest with ts-jest for TypeScript support.

Test File	Tests	Coverage Area
tests/unit/pricing/strategies.test.ts	14	Each strategy: correct price calculation, correct multipliers, boundary conditions for bulk tiers, runtime switching between strategies
tests/unit/pricing/decorators.test.ts	14	Each decorator: correct discount application, composition/stacking of multiple decorators, edge cases (zero-price floor, invalid coupon codes, loyalty tier boundaries)
tests/unit/orders/orderStateMachine.test.ts	8	All valid transitions (isValidTransition returns true), all invalid transitions (throws error), terminal state behaviour, getAvailableTransitions output

8.2 Test Design Philosophy

Unit tests are written against the service and pattern classes directly, without requiring a running server or database connection. This means tests are fast (sub-second) and deterministic. The test structure mirrors the source directory structure, making it easy to identify which tests cover which code.

Integration tests are reserved in the tests/integration/ directory for future implementation, which would cover end-to-end flows including MongoDB interactions and event chain verification.

9. Project Structure

The project follows a domain-driven folder structure where each module (inventory, pricing, orders, payments) is self-contained with its own model, repository, service, controller, and routes file. Shared infrastructure (EventBus, middleware, utils) lives in dedicated directories.

Directory / File	Purpose
src/app.ts	Express application setup and route mounting
src/server.ts	Entry point — starts server, registers all event handlers
src/config/	Environment config (env.ts) and MongoDB connection with retry logic (database.ts)
src/events/EventBus.ts	Singleton EventBus implementation (Observer pattern)
src/events/handlers/inventoryHandlers.ts	Handles ORDER_CREATED (reserve) and ORDER_CANCELLED (release)
src/events/handlers/orderHandlers.ts	Handles INVENTORY_RESERVED (confirm order)
src/inventory/	Full inventory module: model, repository, service, controller, routes
src/pricing/strategies/	RegularPricing, SeasonalPricing, BulkPricing, MemberPricing
src/pricing/decorators/	PercentageDiscount, FlatDiscount, CouponDiscount, LoyaltyDiscount
src/orders/orderStateMachine.ts	State machine with all valid transitions and terminal states
src/orders/idempotencyModel.ts	Mongoose schema for idempotency key storage (24h TTL)
src/payments/adapters/	PaymentAdapter abstract class and StripeAdapter stub
src/middleware/	errorHandler, validateRequest (Joi), rateLimiter, idempotencyMiddleware
src/utils/	responseFormatter, logger (Winston), constants (event names, statuses)
tests/unit/	36 unit tests across pricing strategies, decorators, state machine
docs/ARCHITECTURE.md	Architecture documentation with Mermaid diagrams

10. Future Improvements

The following improvements are identified as natural next steps, in order of priority:

Improvement	Priority	Impact	Notes
Real Stripe Integration	High	Enables live payment processing	The Adapter pattern means only StripeAdapter.ts needs to be implemented — no other code changes
Redis-Backed EventBus	High	Distributed, persistent events	Current in-memory EventBus does not survive restarts. Redis pub/sub or a message queue (RabbitMQ, Kafka) would make it production-grade
WebSocket Order Updates	Medium	Real-time client notifications	Emit WebSocket events on ORDER_CONFIRMED, SHIPPED, DELIVERED so clients receive push updates
Integration Tests	Medium	End-to-end confidence	Test full event chains with a real MongoDB instance using test containers
Authentication & RBAC	Medium	Security	Add JWT authentication; restrict admin endpoints (add product, update stock) to admin role
Dead Letter Queue	Medium	Fault tolerance	If InventoryHandler fails to reserve stock, an unhandled rejection currently goes untracked. A DLQ would capture and retry failed events
OpenAPI / Swagger Docs	Low	Developer experience	Auto-generate API documentation from route definitions
Docker Compose Setup	Low	Developer experience	Single command to spin up app + MongoDB for local development

11. Conclusion

This project demonstrates a production-architecture approach to an e-commerce backend, built entirely as a solo effort across 50 commits. The core thesis — that event-driven architecture eliminates tight coupling between domain modules — is proven by the implementation: OrderService has no imports from InventoryService, yet inventory is correctly reserved, confirmed, and released in response to every order lifecycle event.

The eight design patterns applied are not academic exercises; each addresses a concrete engineering problem. The Strategy and Decorator patterns make pricing extensible without touching consumers. The State Machine prevents invalid order state mutations. The Adapter pattern future-proofs payment integration. The Repository pattern keeps business logic testable without a database.

The 36 passing unit tests demonstrate that each pattern implementation is correct in isolation, covering boundary conditions, composition edge cases, and invalid state transitions.

Key Achievements

- Event-driven architecture with zero inter-service imports
- 8 design patterns applied with clear justification
- Optimistic locking prevents inventory race conditions
- Idempotency middleware prevents duplicate orders
- 36 unit tests, all passing
- Payment adapter ready for real provider integration
- 100% sole contributor across all 50 commits

— End of Report —